Mario Casalini matricola: 7164154
Corso: Architetture degli Elaboratori A.A. 2024/2025 consegna 04/07/2025
Titolo: Gestione di Liste Concatenate in Assembly RISC-V

Relazione Progetto AE 2025 Mario Casalini

Introduzione:

Il progetto vuole elaborare una stringa data in input , listinput, la quale deve rispettare determinate richieste per elaborare le funzioni, la gestione di una lista concatenata non molto complessa. Vieni simulato un comportamento simile a quello di un linguaggio ad alto livello come può essere in C/C++ perché si vuole programmare tutto ciò in risc-v assembly in ripes in un linguaggio a basso livello. Il programma prende un blocco di memoria già inserita manualmente simulando una memoria dinamica con il solo limite della memoria predefinita che in questo caso sono 150 byte quindi 30 nodi per la lista concatenata. La lista concatenata dovrà gestire le funzione di ADD (aggiunta di un nodo) , DEL (eliminazione di uno o più nodi), PRINT (stampa in modo ricorsivo i contenuti di dati all'interno della lista), REV (inverte la lista), SORT (ordina con un bubble sort in modo ricorsivo la lista seguendo dei criteri ben precisi).

Soluzione adottata:

La soluzione adottata è stata la simulazione di una lista concatenata in programmazione risc-v su cui operano i comandi forniti dalla stringa data in input ed è basata a linguaggio a basso livello quindi senza l'utilizzo di funzioni predefinite. Viene inserito un array predefinito dalla memoria_libera e viene gestito manualmente, vengono validate le funzioni carattere per carattere e viene assunto che ogni comando sia una funzione quindi vengono richiamate attraverso le jal (utilizzato per il jump and link, il salto incondizionato a una riga precisa del codice, quindi usando il comando ret in fondo alla funzione ci permette di tornare alla jal e quindi con una j (jump) di tornare nel ciclo madre.

Come viene simulata la memoria: La lista quindi è memorizzata in un array predefinito già in partenza con al massimo 30 nodi ogni nodo ha 5 byte e quindi il totale della memoria usata sono 150 byte. 1 byte per il DATA → contiene un carattere; 4 byte per il PAHEAD → contiene il riferimento del nodo successivo. Il registro s6 in questo caso permette di avere sempre la prossima area libera disponibile in memoria e quindi in caso di add viene usato grazie a s6 il primo posto di memoria utilizzabile e mentre in caso di del i nodi eliminati vengono annullati quindi non avranno più nessun dato e nessun collegamento al loro interno e saranno rimessi in testa alla coda per riutilizzare la memoria e non dispenderla. Sono 30 i nodi perché al massimo le add possibile possono essere 30 quindi non più di 30 nodi per la lista se fosserò più di 30 add il programma non partirebbe.

```
# Allocazione di 150 byte per la memoria dinamica della lista
# Ogni nodo occupa 5 byte: 1 per il carattere DATA, 4 per il puntatore PAHEAD
memoria_libera:
    .byte 0, 0, 0, 0, 0     # Nodo 1
    .byte 0, 0, 0, 0, 0     # Nodo 2
    .byte 0, 0, 0, 0, 0     # Nodo 3
    .byte 0, 0, 0, 0, 0     # Nodo 4
    .byte 0, 0, 0, 0, 0     # Nodo 5
    .byte 0, 0, 0, 0, 0     # Nodo 6
    .byte 0, 0, 0, 0, 0     # Nodo 7
    .byte 0, 0, 0, 0, 0     # Nodo 8
    .byte 0, 0, 0, 0, 0     # Nodo 9
    .byte 0, 0, 0, 0, 0     # Nodo 10
    .byte 0, 0, 0, 0, 0     # Nodo 11
    .byte 0, 0, 0, 0, 0     # Nodo 12
    .byte 0, 0, 0, 0, 0     # Nodo 13
    .byte 0, 0, 0, 0, 0     # Nodo 14
    .byte 0, 0, 0, 0, 0     # Nodo 15
    .byte 0, 0, 0, 0, 0     # Nodo 16
    .byte 0, 0, 0, 0, 0     # Nodo 17
    .byte 0, 0, 0, 0, 0     # Nodo 18
    .byte 0, 0, 0, 0, 0     # Nodo 19
    .byte 0, 0, 0, 0, 0     # Nodo 20
    .byte 0, 0, 0, 0, 0     # Nodo 21
    .byte 0, 0, 0, 0, 0     # Nodo 22
    .byte 0, 0, 0, 0, 0     # Nodo 23
    .byte 0, 0, 0, 0, 0     # Nodo 24
    .byte 0, 0, 0, 0, 0     # Nodo 25
    .byte 0, 0, 0, 0, 0     # Nodo 26
    .byte 0, 0, 0, 0, 0     # Nodo 27
    .byte 0, 0, 0, 0, 0     # Nodo 28
    .byte 0, 0, 0, 0, 0     # Nodo 29
    .byte 0, 0, 0, 0, 0     # Nodo 30
```

Validazione dell'input: viene data in input una stringa linstinput e da questa stringa dobbiamo dedurre le funzioni da applicare all'interno della lista concatenata. quindi viene scorsa la lista, i comandi vengono separati dalla tilde e gli spazi devono essere ignorati e quindi i comandi vengono messi all'interno di un buffer per la validazione questo buffer una volta inserito il comando che sarà all'interno di due tilde sarà inserito nel buffer senza spazi, quindi verranno verificati i comandi e devono avere nel caso di ADD, 6 caratteri e parentesi corrette. ADD e DEL avranno una ulteriore verifica che sarà nel carattere interno che dovrà essere compreso tra 32 e 125 in ASCII. Se il comando è valido il programma effettua una jal alla funzione corrispondente, se non è valido stampa un messaggio di errore e passa al comando successivo, quindi il buffer sarà ripristinato e verrà inserito un nuovo comando.
ecco un possibile input nel programma:

```
# Input principale da analizzare: lista di comandi separati da ~
lista: .string "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(;) ~  ADD(9) ~SORT~PRINT~DEL(b) ~DEL(B) ~PRI~REV~PRIN
```

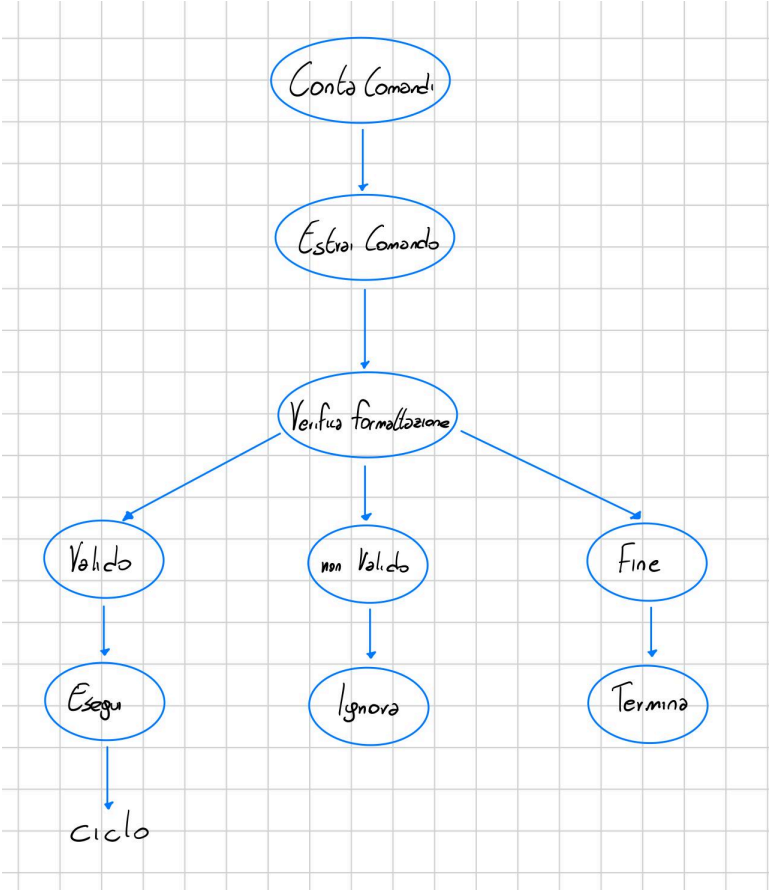
e il buffer per l'allocazione temporanea delle stringhe di funzioni: nel programma viene usato il registro t0 come puntatore per il buffer e i caratteri delle funzioni vengono salvati tramite sb e analizzati da verificaFormattazione.

```
# Buffer per la manipolazione temporanea di stringhe o dati
buffer:
    .byte 0  # 1
    .byte 0  # 2
    .byte 0  # 3
    .byte 0  # 4
    .byte 0  # 5
    .byte 0  # 6
    .byte 0  # 7
    .byte 0  # 8
    .byte 0  # 9
    .byte 0  # 10
    .byte 0  # 11
    .byte 0  # 12
    .byte 0  # 13
    .byte 0  # 14
    .byte 0  # 15
    .byte 0  # 16
    .byte 0  # 17
    .byte 0  # 18
    .byte 0  # 19
    .byte 0  # 20
    .byte 0  # 21
    .byte 0  # 22
    .byte 0  # 23
    .byte 0  # 24
    .byte 0  # 25
    .byte 0  # 26
    .byte 0  # 27
    .byte 0  # 28
    .byte 0  # 29
    .byte 0  # 30
    .byte 0  # 31
    .byte 0  # 32
```

Uso dei registri: I registri vengono usati in modi diversi:

- s0→s7 (registri che servono per tutto il programma): servono per le informazioni che possono servire in tutto il programma e quindi per conservare dati utili come l'indirizzo a un nodo successivo o un puntatore, ecc...
- t0→t6 (registri che non servono per tutto il programma): sono usati all'interno del programma per ciclo o confronti delle funzioni quindi anche se in risc-v i registri non avrebbero dispersione di dati se si cambia funzione non ci serviranno le informazioni precedenti quindi verranno sovrascritti con altre.
- a0 e a1: usati per syscall e chiamate tra le funzioni di stampa
- sp: è definito come lo stack pointer per risc-v ovvero è usato per salvare le chiamate di ritorno quindi con ra e per le chiamate ricorsive PRINT e SORT: per ra viene salvato sullo stack (l'area di memoria puntata di sp) si fa decrementatndo sp con addi sp, sp -X e poi si scriverà ra nella posizione sw ra, offset(sp) → offset identifica l'allocazione di memoria specifica all'interno dello stack di sp. Invece nelle chiamate ricorsive PRINT e SORT si basano molto sullo stack di sp perché ogni volta che avviene la ricorsione viene salvato l'indirizzo di ritorno di ra in modo da tornare sempre al caso base e permette di salvare delle variabili locali qual'ora serviresso nei registri s che poi possono essere modificati nei livelli successivi della ricorsione quindi ogni livello ha il proprio spazio di dati dove è possibile accederci e accedere a altri dati

diagramma del funzionamento generale del programma:



inoltre nel .data sono presenti le stringhe che permettono di capire cosa sta succedendo all'interno del programma, per la validazione il comando valido o non valido e l'ouput della PRINT. Stampate medianet syscall (ecall) con a0 contenente l'indirizzo della stringa → la a0,nome_stringa.

```
# Vari messaggi di output utilizzati durante l'esecuzione

verifica:           .string     "\nControllo formattazione "
comandoValido:      .string     "\nComando Valido \n"
comandoNonValido:   .string     "\nComando Non Valido \n"
eliminato:          .string     "\nElemento trovato ed eliminato \n"
nonTrovato:         .string     "\nElemento non trovato \n"
ordinata:           .string     "\nLista ordinata\n"
stampa:             .string     "\nStampa: "
invertita:          .string     "\nLista invertita correttamente"
inputCorretto:      .string     "\nNumero di comandi validi \n"
erroreInput:        .string     "\nNumero di comandi maggiore o uguale a 30 \n"
erroreMemoriaPiena: .string    "\nMemoria piena, elemento non aggiunto \n"
endS:               .string     "\nProgramma terminato"
newLine:            .string     "\n"                # Nuova linea per formattazion
```

come viene trovato un blocco libero in memoria: quando si deve aggiungere un nuovo nodo si deve trovare una zona all'interno della memoria_libera, avviene usando il registro s6, che punta sempre all'indirizzo del prossimo nodo disponibile

```
# 2. Controlla se c'è memoria libera
li t2, 0
beq s6, t2, memoria_piena    # Se s6 (freelist) == 0, memoria esaurita

# 3. Alloca un nuovo nodo: usa il blocco puntato da s6
mv t2, s6              # t2 = indirizzo nuovo nodo
addi s6, s6, 5         # Avanza la freelist al prossimo nodo disponibile
```

s6 contiene l'indirizzo del primo nodo libero in memoria_libera, questo indirizzo viene salvato in t2, che sarà usato per scrivere il nuovo nodo, a s6 verranno aggiunti 5 byte per puntare al prossimo nodo che è lo spazio disponibile subito dopo. viene eseguito cosi perchè è una memoria continua e quindi se aggiungo 5 byte a s6 salterà da un nodo all'altro. Se s6 supera la memoria di 150 byte viene rilevato e stampato la stringa di memoria piena:

```
memoria_piena:
    # In caso di memoria piena, stampa messaggio e termina funzione
    la a0, erroreMemoriaPiena   # Messaggio: "Memoria piena, elemento non aggiunto"
    li a7, 4
    ecall
    j fine_add
```

come viene reso un blocco nuovamente libero: quando viene eliminato un nodo con la DEL, il suo spazio sarà di nuovo accessibile senza perdere alcun byte in memoria e quindi il pieno utilizzo di tutti. viene fatto attraverso queste righe:

```
sb zero, 0(t2)          # Reset DATA
addi t6, t2, 1
sw zero, 0(t6)          # Reset PAHEAD
```

queste righe di codice permettono di inserire il nodo da eliminare (t2) in testa alla lista, elimina la sezione data riportandola a zero e collega il nodo alla lista dei nodi liberi prendendo il precedente e il successivo collegandoli insieme t2 si troverebbe staccato allora viene collegato t2 al primo nodo della lista libera e s6 assumerà t2 cosi da mettere in testa t2 alla lista dei nodi liberi.

Inizializzazione del main: viene pulito l'ouput stampando \n per mandare una riga vuota

```
# Stampa una nuova linea per separazione visiva (pulizia iniziale output)
la a0, newLine      # Carica indirizzo della stringa di newline
li a7, 4            # Codice syscall per stampa stringa
ecall               # Stampa newline
```

successivamente vengono inizializzati i registri principali

```
# Inizializzazione dei registri
la s0, lista            # s0 punta alla stringa lista (input dei comandi)
li sp, 0x10000          # inizializza lo stack pointer (es. base sicura)
li s1, 0                # s1: contenuto DATA del nodo corrente
li s2, 0x00000000       # s2: PAHEAD (puntatore al prossimo nodo)
li s3, 0                # s3: tail della lista (coda)
li s4, 0                # s4: contatore dei comandi letti
li s5, 0                # s5: flag errore (0 = ok, 1 = errore)
la s6, memoria_libera   # s6: inizio area di memoria per nodi (freelist)
```

conteggio comandi: vengono contati quanti comandi totali sono presenti all'interno della stringa listinput, ogni comando è separato dalla tilde quindi vengono contate quante tilde ci sono, inizializzando un contatore dei comandi e un registro con il codice ascii per il confronto delle tilde

```
contaComandi:
    li s4, 0            # Reset contatore comandi
    li t0, 126          # Codice ASCII del carattere '~'
```

verrà fatto un ciclo del conteggio leggendo ogni carattere della stringa se è tilde lo conta come separatore senno se è la fine della stringa \0 terminerà di contare

```
ciclo:
    lb t1, 0(s0)          # Legge carattere corrente da lista
    beqz t1, fineConta    # Se è terminatore (0), fine conteggio
    beq t1, t0, tilde     # Se è tilde, vai ad aggiornare contatore
    addi s0, s0, 1        # Altrimenti, passa al carattere successivo
    j ciclo
```

se viene trovata una tilde aumenta s4 per il conteggio se è maggiore o uguale a 30 verrà eseguito l'errore

```
tilde:
    addi s4, s4, 1        # Incrementa il numero di comandi
    addi s0, s0, 1        # Passa oltre la tilde
    li t2, 30             # Limite massimo di comandi
    bge s4, t2, troppi    # Se >=30, mostra errore e termina
    j ciclo
```

quindi se vengono trovati più di 30 comandi verranno gestiti dalla sezione troppi dove il programma non procederà con l'esecuzione e stamperà il messaggio di errore andando nella sezione finale di end

```
troppi:
    li s5, 1                  # Imposta flag errore
    la a0, erroreInput        # Messaggio di errore per troppi comandi
    li a7, 4
    ecall
    j end                     # Salta alla fine del programma
```

se il conteggio è minore di 30 verrà stampato in output il messaggio di conteggio dei comandi e il suo numero

```
fineConta:
    la a0, inputCorretto      # Messaggio: numero comandi valido
    li a7, 4
    ecall
    mv a0, s4                 # Mostra quanti comandi sono stati contati
    li a7, 1                  # Stampa numero intero
    ecall
```

se i comandi sono stati contati si ricomincia da capo per l'analizzazione e la loro esecuzione

```
la s0, lista                  # Ripristina s0 all'inizio della lista
j whileHasChar                # Avvia parsing ed esecuzione comandi
```

Preparazione dei comandi: questa sezione del programma serve per leggere ogni singolo comandi uno a uno dalla stringa lintinput metterli nel buffer e fargli la verifica se sono validi o no. la sezione parte con l'inizializzazione del registro s0 per leggere carattere per carattere la stringa listinput, se arriva a \0 termina saltando alla fine del programma. i registri s0 sarà per puntatore alla stringa e il registro t0 per prendere il carattere del puntatore della stringa

```
lb t0, 0(s0)                  # Legge primo carattere della lista
beqz t0, end                  # Se fine stringa, termina programma
```

quindi per estrarre il comando e metterlo nel buffer si carica t1 come buffer

```
la t1, buffer            # t1 punta all'inizio del buffer per il comando
```

si passa all'estrazione del comando dalla stringa lista fino a che non trova la tilde o \0, il registro t2 è usato per l'estrazione carattere per carattere e t3 per la tilde

```
getCmd:
    lb t2, 0(s0)             # Leggi carattere corrente
    beqz t2, fineGet         # Se è null terminator, fine comando
    li t3, 126               # Codice ASCII di '~'
    beq t2, t3, fineGet      # Se è '~', fine comando
```

quindi una volta caricato il comando in t2 si prende e si mette nel buffer saltando gli spazi e soltanto i caratteri vengono copiati nel buffer. Il registro t4 viene usato per il confronto con ascii 32 (spazio) e il t1 come puntatore interno del buffer

```
li t4, 32                # Codice ASCII per spazio
beq t2, t4, salta_spazi   # Se è spazio, ignoralo
sb t2, 0(t1)             # Altrimenti, salva nel buffer
addi t1, t1, 1           # Avanza nel buffer
```

per saltare gli spazi viene usata la sezione si salto spazio:

```
salta_spazi:
    addi s0, s0, 1           # Avanza nella stringa lista
    j getCmd
```

alla fine viene chiuso corretamente la stringa nel buffer con il terminatore di stringa

```
fineGet:
    sb zero, 0(t1)           # Terminatore di stringa nel buffer
```

per il controllo degli spazi tra comandi e tilde è stato messo un controllo di salto spazi dopo la tilde

```
salta_spazi_dopo_tilde:
    lb t3, 0(s0)             # Leggi carattere successivo
    li t4, 32                # Spazio
    beq t3, t4, salta_spazi_dopo
    li t4, 126               # Tilde
    beq t3, t4, salta_tilde
    beqz t3, ultimaIstruzione  # Se è null termina, è l'ultimo comando
    j verificaFormattazione    # Altrimenti va a verificare il comando
```

per il salto di spazi dopo il comando fino alla tilde vengono usate le sezioni di salto spazio e salto tilde

```
salta_spazi_dopo:
    addi s0, s0, 1
    j salta_spazi_dopo_tilde

salta_tilde:
    addi s0, s0, 1
    j salta_spazi_dopo_tilde
```

per controllare l'ultimo comando senza la tilde finale viene messa questa ultimaistruzione:

```
ultimaIstruzione:
    j verificaFormattazione  # Verifica comunque l'ultimo comando prima di terminare
```

verificazione validità dei comandi: in questa parte del programma si controlla se il comando inserito nel buffer sia valido e successivamente indirizzarlo verso la sua funzione oppure se il comando non è valido restituire in output la stringa comando non valido, come prima cosa quindi si mostra in output che si sta controllando la formattazione per mostrare anche all'utente che è entrato nel processo di verifica

```
# Stampa messaggio per iniziare la validazione del comando
la a0, verifica        # Messaggio: "Controllo formattazione"
li a7, 4
ecall
```

successivamente vengono caricati i caratteri dal buffer per confrontarli con quelli che si dovrebbe aspettare

```
# Caricamento dei primi 7 caratteri (fino a 6 lettere + terminatore) del comando nel regi
lb t1, 0(t0)          # t1 = primo carattere
lb t2, 1(t0)          # t2 = secondo carattere
lb t3, 2(t0)          # t3 = terzo carattere
lb t4, 3(t0)          # t4 = carattere di apertura
lb t5, 4(t0)          # t5 = carattere parametro (es. 'a')
lb t6, 5(t0)          # t6 = carattere di chiusura
lb s11, 6(t0)         # s11 = dovrebbe essere null terminator
```

si verifica quindi se è il comando di ADD controllando anche il carattere dentro le parentesi

```
# Verifica se è un comando "ADD(?)"
li s10, 65            # 'A'
bne t1, s10, cotrolloDEL
li s10, 68            # 'D'
bne t2, s10, cotrolloDEL
bne t3, s10, cotrolloDEL
li s10, 40
bne t4, s10, cotrolloDEL
li s10, 41
bne t6, s10, cotrolloDEL
beqz s11, controlloIntervallo  # se fine stringa, controlla validità del carattere parame
j cmdNonValido
```

per controllare il carattere dentro le parentesi si usa il controlloIntervallo per verificare che sia compreso tra 32 e 125 in codice ASCII, se il carattere è compatibile il comando sarà valido

```
controlloIntervallo:
    # Verifica se il parametro t5 è un carattere valido tra 32 e 125 ASCII
    li s9, 32
    blt t5, s9, cmdNonValido
    li s9, 125
    bgt t5, s9, cmdNonValido
    j cmdValido
```

viene fatto anche il controllo per se è una DEL

```
controlloDEL:
    # Verifica se è un comando "DEL(?)"
    li s10, 68            # 'D'
    bne t1, s10, controlloPRINT
    li s10, 69            # 'E'
    bne t2, s10, controlloPRINT
    li s10, 76            # 'L'
    bne t3, s10, controlloPRINT
    li s10, 40
    bne t4, s10, controlloPRINT
    li s10, 41
    bne t6, s10, controlloPRINT
    beqz s11, controlloIntervallo  # se termina correttamente, controlla carattere
    j cmdNonValido
```

viene ripetuto uguale per la print rev e sort a differenza che non avranno il controllo dell'intervallo tra parentesi, se si arriva in fondo il comando sarà valido perché non ci saranno errori di controllo quindi si può stampare il messaggio di comando valido

```
cmdValido:
    # Stampa messaggio di validità del comando
    la a0, comandoValido
    li a7, 4
    ecall
```

e quindi essendo tutte funzioni con iniziali diverse si controlla soltanto il primo carattere, quindi se c'è la A si manda alla chiamata della add, se c'è la d si manderà alla chiamata della DEL e cosi per le altre

```
# Identifica il comando e chiama la funzione corretta
lb t0, buffer          # Primo carattere per determinare il comando

li t1, 65              # 'A' → ADD
beq t0, t1, is_add

li t1, 68              # 'D' → DEL
beq t0, t1, is_del

li t1, 80              # 'P' → PRINT
beq t0, t1, is_print

li t1, 83              # 'S' → SORT
beq t0, t1, is_sort

li t1, 82              # 'R' → REV
beq t0, t1, is_rev

j whileHasChar         # Se non è nessuno di questi, continua
```

se il comando non passa i controlli viene mandato in output la stringa di comando non valido

```
cmdNonValido:
    # Stampa messaggio di errore e ignora comando
    la a0, comandoNonValido
    li a7, 4
    ecall
    j whileHasChar          # Passa al prossimo comando
```

le chiamate alle funzioni vengono fatte attraverso questa sezione di codice che chiama con una jal la funzione e poi ritorna al ciclo per continuare l'esecuzione

```
# Chiamate alle procedure
is_add:
    jal addElemento        # Chiama procedura per inserimento
    j whileHasChar

is_del:
    jal delElemento        # Chiama procedura per eliminazione
    j whileHasChar

is_print:
    jal printLista         # Chiama procedura per stampa lista
    j whileHasChar

is_sort:
    jal sortListaRicorsiva # Chiama procedura ricorsiva per ordinamento
    j whileHasChar

is_rev:
    jal revLista           # Chiama procedura di inversione
    j whileHasChar
```

I registri utilizzati sono : t0 per il puntatore al buffer, da t1 → t6 vengono caricati i caratteri da verificare; s10 serve per il confronto del codice ascii; s11 viene usato per \0 terminatore di stringa e s9 per il limite di ascii 32 a 125

Funzione add: la funzione add viene chiamata quando il comando è valido e viene usata per inserire un nuovo nodo nella memoria simulata , memoria_libera, inserisce il carattere dentro le parentesi all'interno della sezione data e successivamente farà il collegamento con un nuovo nodo. Si prende il carattere al centro del comando ADD, prima il registro ra viene messo nello stack per la chiamata ret alla fine e non disperdere ra

```
# === Funzione ADD ===
addElemento:
    # Salva contesto (ra) sullo stack per non perdere return address
    addi sp, sp, -4
    sw ra, 0(sp)      # Salva il return address
```

posso leggere il carattere all'interno delle parentesi

```
# 1. Leggi il carattere da inserire dal buffer (alla posizione 4 corrispondente a ADD(x))
la t0, buffer
lb t1, 4(t0)           # t1 = DATA, il carattere da salvare nel nodo
```

si controlla se la memoria libera è piena con il controllo a s6 → puntatore alla memoria libera, se la memoria è piena si stampa il messaggio di errore

```
# 2. Controlla se c'è memoria libera
li t2, 0
beq s6, t2, memoria_piena    # Se s6 (freelist) == 0, memoria esaurita
```

quindi se la memoria non è piena si alloca un nuovo nodo per il DATA e il PAHEAD e si sposta s6 al nodo successivo

```
# 3. Alloca un nuovo nodo: usa il blocco puntato da s6
mv t2, s6              # t2 = indirizzo nuovo nodo
addi s6, s6, 5         # Avanza la freelist al prossimo nodo disponibile
```

quindi si carica nel primo byte il carattere dentro la ADD e si mette nel nodo e si inizializza inizialmente il PAHEAD a 0 e controllo se mettere il nodo in coda o in prima posizione

```
# 4. Inizializza il nuovo nodo
sb t1, 0(t2)           # [0] = DATA: salva il carattere all'inizio del nodo
sw zero, 1(t2)         # [1-4] = PAHEAD: inizializza puntatore a 0 (ultimo elemento per o

# 5. Se la lista è vuota (head == 0), inizializza anche la testa
li t3, 0
beq s2, t3, lista_vuota   # s2 = head; se 0, la lista è vuota

# 6. Altrimenti, collega il nuovo nodo in fondo alla lista
sw t2, 1(s3)           # s3 = tail; tail->PAHEAD = nuovo nodo
j trovata_testa
```

se la lista è vuota il nodo diventa la testa

```
lista_vuota:
    mv s2, t2              # s2 = head = nuovo nodo (inizializza testa della lista)
```

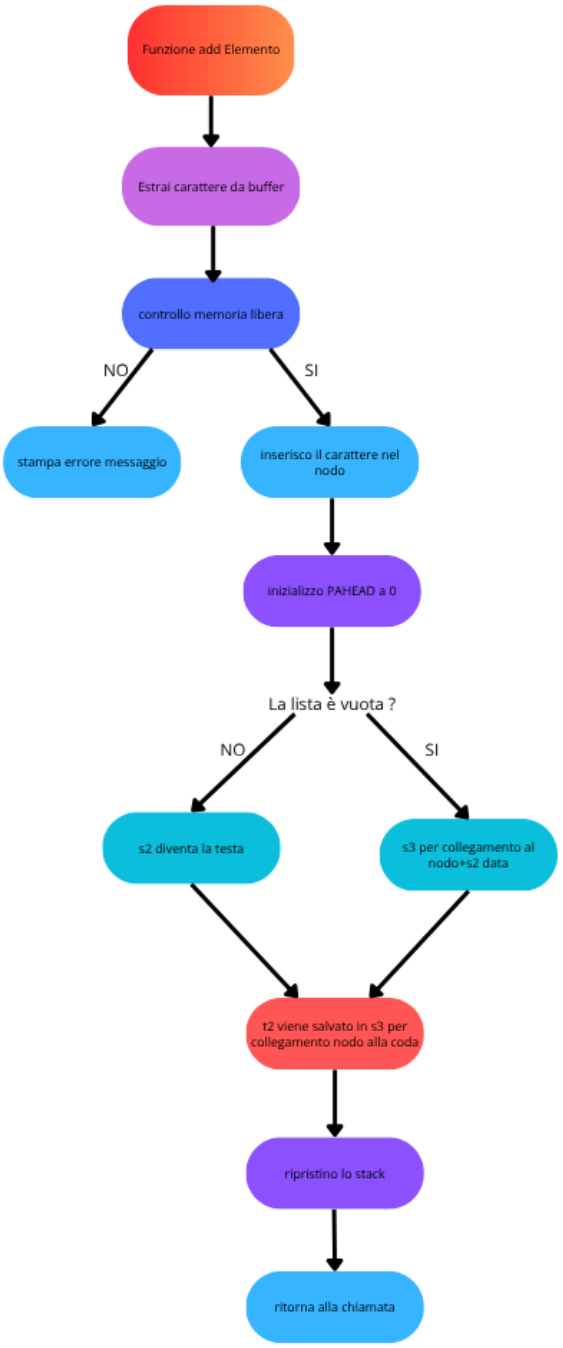
per l'inserimento in coda si usa senno

```
trovata_testa:
    mv s3, t2              # s3 = tail = nuovo nodo (aggiorna coda della lista)
```

quindi si ripristina lo stack e ritorna alla chiamata jal tramite la ret che prendere l'indirizzo da ra (return address)

```
fine_add:
    # Ripristina il contesto salvato e torna alla funzione chiamante
    lw ra, 0(sp)
    addi sp, sp, 4
    ret
```

I registri usati sono: t0 per il puntatore al buffer; t1 per il carattere DATA dentro la ADD; t2 per l'indirizzo del nuovo nodo; t3 è il registro temporaneo dei confronti; s2 la testa della lista; s3 la coda della lista; s6 il puntatore alla memoria libera ; e ra il return address salvato nello stack.
Flowchart che spiega come funziona:



Funzione DEL: la funzione DEL rimuove tutti i nodi che hanno un determinato carattere all'interno delle parentesi, si prende il carattere da eliminare si visita ogni nodo e si confronta l'elemento con il DATA se è uguale lo elimina e rimettete nella sezione di memoria libera il nodo e collega i nodi per mantenere la lista concatenata. Come per la add salvo l'indirizzo ra prima di fare tutto

```
# === Funzione DEL (rimozione di tutti i nodi con lo stesso DATA) ===
delElemento:
    # Salva indirizzo (RA)
    addi sp, sp, -4
    sw ra, 0(sp)
```

Si carica il carattere da rimuovere in t1 con t2 si prende la testa e si scorre la lista salvando sempre il nodo precedente in t4

```
# Estrai il carattere da eliminare dal buffer (es. DEL(a))
la t0, buffer
lb t1, 4(t0)          # t1 = carattere target

mv t2, s2             # t2 = nodo corrente = testa
li t4, 0              # t4 = nodo precedente (null)

li s7, 0              # flag = 0 → nessun nodo eliminato finora
```

si effettua un ciclo per scorrere la lista, se non corrisponde va avanti e aggiorna il precedente

```
ciclo_del:
    beqz t2, stampaStringa_del      # Se nodo corrente è nullo, fine lista

    lb t3, 0(t2)          # t3 = DATA del nodo
    addi t6, t2, 1        # t6 = indirizzo di PAHEAD
    lw t5, 0(t6)          # t5 = PAHEAD = prossimo nodo

    bne t3, t1, prossimo_nodo  # Se il dato non corrisponde, vai avanti

    # Nodo da eliminare
    beqz t4, del_testa      # Se precedente = 0 → nodo in testa
```

successivamente se si trova il nodo da eliminare si elimina e lo si mette in testa nella lista di memoria libera, se viene eliminato almeno un nodo s7 va a 1

```
continua_del:
    addi t6, t4, 1
    sw t5, 0(t6)          # precedente->PAHEAD = prossimo

    # Rimuovi nodo t2 reinserendolo nella freelist
    sb zero, 0(t2)        # resetto DATA
    sw s6, 1(t2)          # t2->PAHEAD = vecchia testa della freelist
    mv s6, t2             # testa della memoria non occupata = t2

    li s7, 1              # Almeno un nodo eliminato
    mv t2, t5             # Passa al prossimo
    j ciclo_del
```

per la rimozione della testa il nodo successivo sarà la nuova testa

```
del_testa:
    mv s2, t5               # Nuova testa = prossimo
    sb zero, 0(t2)          # Reset DATA
    addi t6, t2, 1
    sw zero, 0(t6)          # Reset PAHEAD
    li s7, 1                # Nodo eliminato
    mv t2, t5
    j ciclo_del
```

se non c'è un nodo da eliminare si va al prossimo

```
prossimo_nodo:
    mv t4, t2               # precente     = corrente
    mv t2, t5               # corrente = prossimo
    j ciclo_del
```

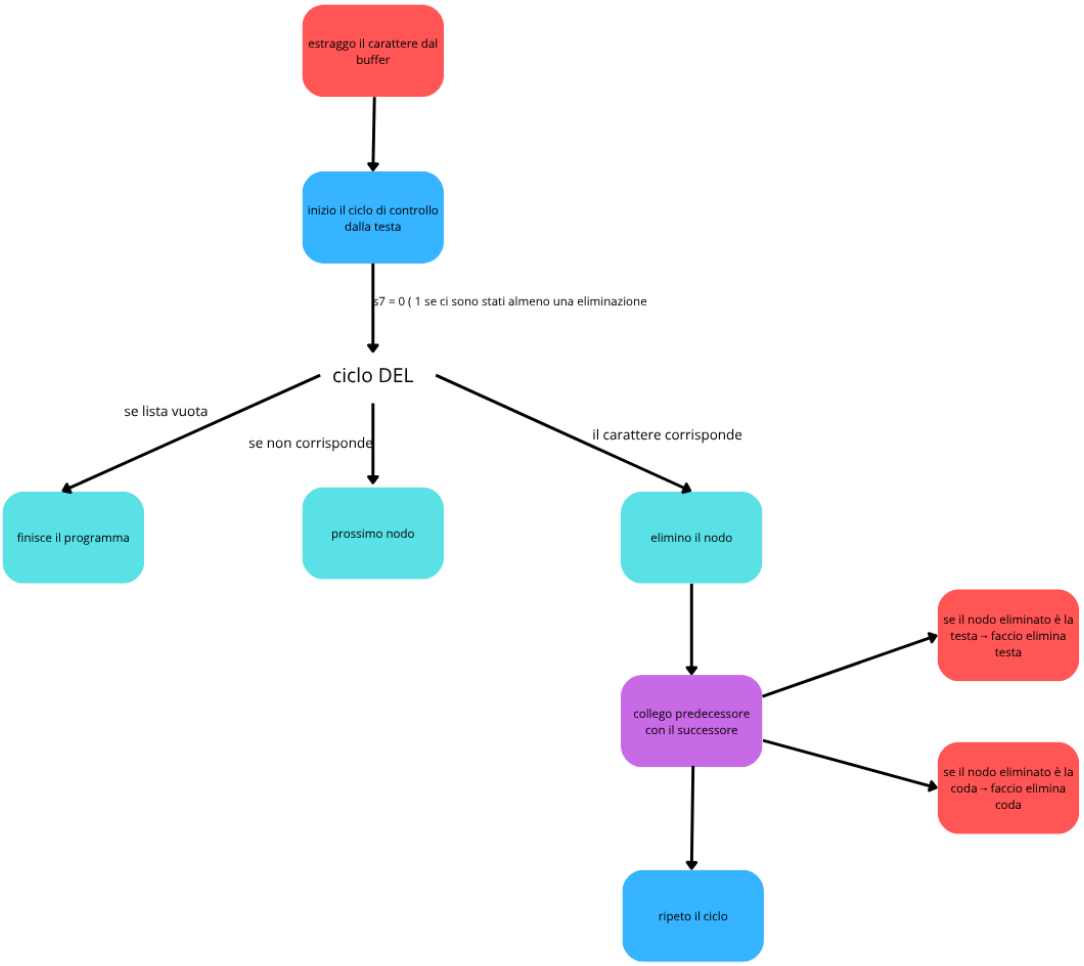
alla fine si stampa se è stato eliminato qualcosa o no tramite il controllo a s7

```
stampaStringa_del:
    beqz s7, stampaNonTrovato    # Se nessun nodo eliminato, messaggio dedicato
    la a0, eliminato
    li a7, 4
    ecall
    j fine_endDel

stampaNonTrovato:
    la a0, nonTrovato
    li a7, 4
    ecall
```

Ripristino lo stack

```
fine_endDel:
    # Ripristina contesto
    lw ra, 0(sp)
    addi sp, sp, 4
    ret
```

I registri usati sono: t0 per il puntatore al buffer; t1 per il carattere da eliminare; t2 per il nodo che viene controllato; t3 per il DATA del nodo in t2; t4 per il nodo precedente; t5 per il nodo successivo; t6 per l'indirizzo PAHEAD; s2 per la testa della lista; s3 per la coda della lista ; s6 per il puntatore alla memoria libera; s7 se 0 nessun nodo eliminato se 1 almeno un nodo eliminato. Flow chart:



Funzione Print: la funzione deve stampare tutti i caratteri dei nodi della lista. viene salvato ra nello stack per mantenere il registro al caso base senza dispersioni o modificazioni, quindi ogni ret torna alla chiamate precedente

```
# === Stampa la lista collegata partendo da head ===
printLista:
    # Salva contesto minimo
    addi sp, sp, -12
    sw ra, 0(sp)
    sw t0, 4(sp)
    sw t1, 8(sp)
```

si inizializza il puntatore t0 alla testa della lista stampando una nuova linea dopo tutta la lista

```
mv t0, s2              # t0 = head
jal printNodoRic       # chiamata ricorsiva

# Dopo stampa: stampa newline
li a7, 11              # syscall: stampa char
li a0, 10              # newline (ASCII 10)
ecall
```

la funzione principale ricorsiva con il richiamo a se stessa finche non torna al caso base passando a ogni chiamata il nodo successivo, si ferma solo quando t0 è zero e quindi si ha la fine della lista

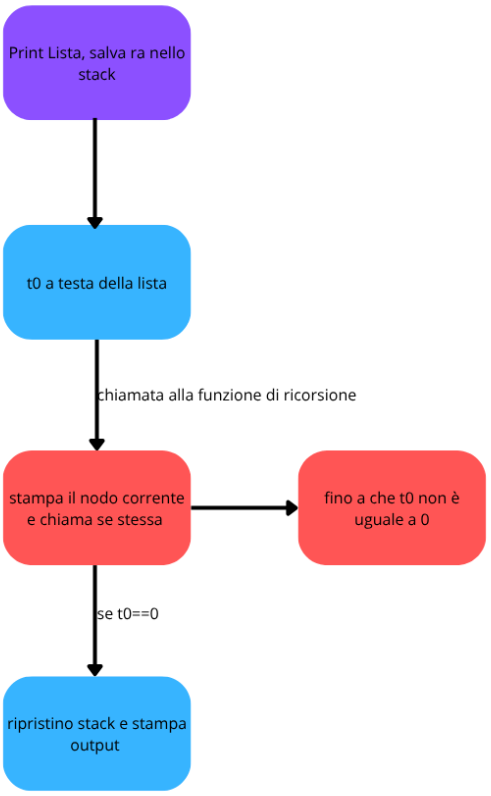
```
printNodoRic:
    # Prologo: salva RA
    addi sp, sp, -8
    sw ra, 0(sp)

    beqz t0, fine          # Caso base: se t0 == 0, fine lista

    lb a0, 0(t0)           # a0 = DATA
    li a7, 11              # syscall stampa char
    ecall

    lw t1, 1(t0)           # t1 = PAHEAD
    mv t0, t1              # t0 = prossimo nodo
    jal printNodoRic       # chiamata ricorsiva

fine:
    # Epilogo: ripristina RA
    lw ra, 0(sp)
    addi sp, sp, 8
    ret
```

alla fine si ripristina lo stack

```
fine:
    # Epilogo: ripristina RA
    lw ra, 0(sp)
    lw t0, 4(sp)
     lw t1, 8(sp)
    addi sp, sp, 12
    ret
```

'

I registri usati sono: s2 puntatore alla testa della lista; t0 come nodo corrente (passato alla ricorsione); t1 puntatore al nodo successivo; a0 contenuto da stampare; a7 codice syscall; ra return address; sp puntatore allo stack. Flow chart

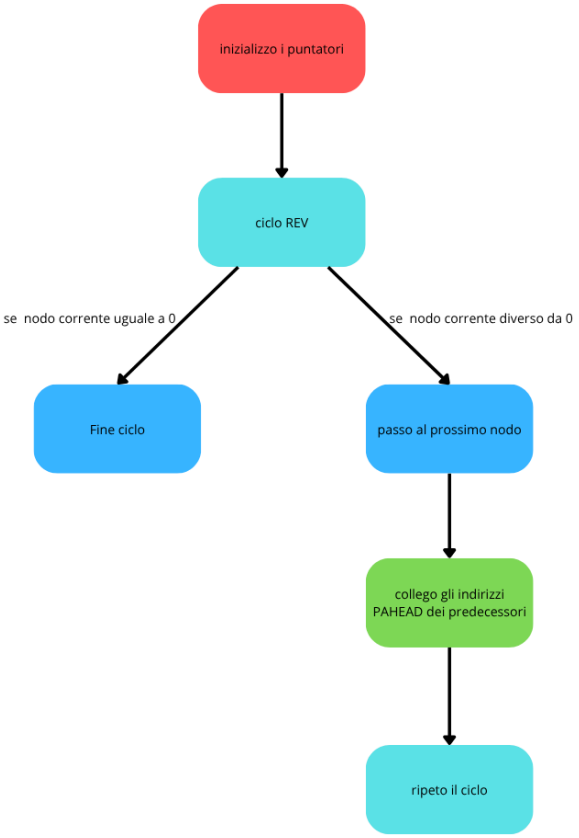


-Funzione REV: la funzione inverte la lista concatenata modificando i campi PAHEAD inizializzando un nodo come corrente e predecessore e gli inverte, quindi se prendo il primo elemento la testa il precedente sarà null→ prendo B che ha come predecessore A e cosi via fino alla coda della lista che diventerà la testa

```
revLista:
    # Salvataggio indiirizzo
    addi sp, sp, -4
    sw ra, 0(sp)

    # Inizializzazione
    mv t0, s2        # t0 = corrente → nodo corrente (parte dalla testa)
    li t1, 0         # t1 = precedente → nodo precedente (inizialmente NULL)

rev_ciclo:
    beqz t0, rev_fine     # Se corrente == NULL (fine lista), uscita

    # Ottieni indirizzo e valore del campo PAHEAD (prossimo nodo)
    addi t2, t0, 1        # t2 = indirizzo di corrente->PAHEAD
    lw t3, 0(t2)          # t3 = prossimo → PAHEAD corrente

    # Inversione puntatore
    sw t1, 0(t2)          # corrente->PAHEAD = precedente (inversione link)

    # Avanza nella lista
    mv t1, t0             # precedente = corrente
    mv t0, t3             # corrente = prossimo
    j rev_ciclo

rev_fine:
    # Fine ciclo: nuova testa è l'ultimo nodo visitato
    mv s2, t1             # s2 = head = prev (nuovo primo nodo)

    # Stampa messaggio di successo
    la a0, invertita      # "Lista invertita correttamente"
    li a7, 4
    ecall

    # Stampa newline
    li a7, 11
    li a0, 10
    ecall

    # Ripristina contesto
    lw ra, 0(sp)
    addi sp, sp, 4
    ret
```

I registri usati sono: s2 testa della lista (viene aggiornato alla fine della funzione) ; t0 usato come nodo corrente; t1 usato come nodo predecessore; t2 come indirizzo PAHEAD del nodo corrente; t3 come

indirizzo PAHEAD del nodo successivo ; a0 per la stampa dei messaggi ; a7 per syscall; sp per il puntatore allo stack; ra come return address. Flow chart



Funzione SORT: viene ordinata la lista seguendo le categorie ASCII quindi in ordine: simboli, cifre, lettere minuscole e maiuscole, considerando che se due elementi hanno la stessa categoria si confrontano i caratteri ascii. Si inizializza una funzione sortListaRicorsiva che è quella principale dato che richiama il bubble sort finché non ci saranno più scambi, viene usato lo stack per ra per non perdere il ra di ogni chiamata

```
sortListaRicorsiva:
    addi sp, sp, -8
    sw ra, 0(sp)          # Salva return address

    jal bubblePass        # Chiama una passata
    beqz a0, fine_sort       # Se non ci sono stati swap, fine
    jal sortListaRicorsiva  # Altrimenti, ripeti ricorsivamente

fine_sort:
    lw ra, 0(sp)
    addi sp, sp, 8
    jr ra                 # Ritorna al chiamante
```

la funzione bubble sort sarà quella richiamata inizializzando lo stack e t0 per il nodo corrente e t1 per nodo successivo, viene fatto un controllo se c'è solo un nodo o la lista è vuota finisce
I registri usati sono:sp per puntatore allo stack; ra per il return address; a0 riceve il valore di ritorno da bubble pass

```
bubblePass:
    addi sp,sp,-16
    sw ra,0(sp)
    sw s0,4(sp)    # flag scambio
    sw s1,8(sp)
    sw s2,12(sp)

    li s0,0        # s0 = flag scambio
    mv t0,s2       # t0 = nodo corrente (inizia da head)
    lw t1,1(t0)    # t1 = prossimo nodo (PAHEAD)
    beqz t1,bp_fine # Se lista vuota o singolo nodo
```

sennò si prosegue verso il ciclo principale della bubble sort dove si estraggono i DATA dei nodi si calcola la categoria attraverso l'indirizzo di categoria e si confrontano, nel caso vengono scambiati se la categoria del secondo è minore del primo

```
bp_ciclo:
    lb t2,0(t0)    # t2 = DATA nodo corrente
    lb t3,0(t1)    # t3 = DATA nodo successivo

    # Calcola categoria di t2
    mv a0,t2
    jal categoria
    mv s1,a1       # categoria nodo corrente

    # Calcola categoria di t3
    mv a0,t3
    jal categoria
    mv s2,a1       # categoria nodo successivo

    # Ordina in base a categoria
    blt s1,s2,bp_prossimo   # ok, non serve scambio
    bgt s1,s2,do_scambio   # scambia

    # Se categoria uguale, confronta ASCII
    bgt t2,t3,do_scambio
    j bp_prossimo

do_scambio:
    sb t3,0(t0)     # nodo corrente = t3
    sb t2,0(t1)     # nodo successivo = t2
    li s0,1         # è avvenuto uno swap
```

per proseguire con i nodi e la fine del bubble sort

```
bp_prossimo:
    lw t4,1(t1)     # t4 = next->PAHEAD
    beqz t4,bp_fine
    mv t0,t1        # t0 = t1 (scorre avanti)
    mv t1,t4
    j bp_ciclo

bp_fine:
    mv a0,s0        # restituisci flag swap
    lw s2,12(sp)
    lw s1,8(sp)
    lw s0,4(sp)
    lw ra,0(sp)
    addi sp,sp,16
    ret

bp_finito:
    li a0,0         # Lista già ordinata
    lw s2,12(sp)
    lw s1,8(sp)
    lw s0,4(sp)
    lw ra,0(sp)
    addi sp,sp,16
    ret
```

I registri usati sono: sp per salvare ra,ì s0 s2 e s2; s0 se 0 nessun scambio se 1 almeno uno scambio avvenuto; s1 categoria nodo corrente; s2 categoria nodo successivo; t0 puntatore nodo corrente; t1 puntatore nodo successivo; t2 valore ASCII del nodo corrente; t3 valore ASCII del nodo successivo; t4 puntatore al nodo successivo ; a0 usato per chiamare categoria; a1 per output categoria

La categoria serve a capire di quale categoria fanno parte e quindi un primo controllo per lo scambio, controlla i DATA dei nodi e i loro bit ascii

```
categoria:
    # Controllo se è cifra: '0' (48) ≤ a0 < '9'+1 (58)
    li t5, 48
    blt a0, t5, cat_extra      # a0 < 48 → simbolo extra
    li t5, 58
    blt a0, t5, cat_digit      # a0 < 58 → è cifra

    # Controllo se è lettera maiuscola: 'A' (65) ≤ a0 < 'Z'+1 (91)
    li t5, 65
    blt a0, t5, cat_extra      # a0 < 65 → simbolo extra
    li t5, 91
    blt a0, t5, cat_upper      # a0 < 91 → è maiuscola

    # Controllo se è lettera minuscola: 'a' (97) ≤ a0 < 'z'+1 (123)
    li t5, 97
    blt a0, t5, cat_extra      # a0 < 97 → simbolo extra
    li t5, 123
    blt a0, t5, cat_lower      # a0 < 123 → è minuscola

# Se non è nessuna delle precedenti, è simbolo extra
cat_extra:
    li a1, 0                   # Categoria 0 = simbolo
    ret

cat_digit:
    li a1, 1                   # Categoria 1 = numero
    ret

cat_lower:
    li a1, 2                   # Categoria 2 = minuscola
    ret

cat_upper:
    li a1, 3                   # Categoria 3 = maiuscola
    ret
```

I registri usati sono: a0 per codice ASCII del carattere; a1 categoria carattere
una volta eseguito tutto si manda in output la stringa di fine e si finisce il programma

```
end:
    la a0, endS
    li a7, 4
    ecall

    li a7, 10      # exit
    ecall
```

TEST:

test1:

stringa : lista: .string "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(;) ~ ADD(9) ~SORT~PRINT~DEL(b) ~DEL(B) ~PRI~REV~PRINT"

stampa attesa:

Mario Casalini matricola: 7164154
Corso: Architetture degli Elaboratori A.A. 2024/2025 consegna 04/07/2025
Titolo: Gestione di Liste Concatenate in Assembly RISC-V

listInput = "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(;) ~ ADD(9) ~SORT~PRINT~DEL(b) ~DEL(B) ~PRI~REV~PRINT"

Comando Corrente	ADD(1)	ADD(a)	ADD(a)	ADD(B)	ADD(;)	ADD(9)	SORT	PRINT	DEL(b)	DEL(B)	PRI	REV	PRINT
Elementi in Lista	1	1a	1aa	1aaB	1aaB;	1aaB;9	;19aaB	;19aaB	;19aaB	;19aa	;19aa	aa91;	aa91;

<u>risultato:</u>

```
Numero di comandi validi
12
Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido
;19aaB

Controllo formattazione
Comando Valido

Elemento non trovato
```

```
Controllo formattazione
Comando Valido

Elemento trovato ed eliminato

Controllo formattazione
Comando Non Valido
Controllo formattazione
Comando Valido

Lista invertita correttamente

Controllo formattazione
Comando Valido
aa91;

Programma terminato
Program exited with code: 0
```

test2:

stringa : listinput: .string "ADD(1) ~ ADD(a) ~ add(B) ~ ADD(B) ~ ADD ~ ADD(9) ~PRINT~SORT(a)~PRINT~DEL(bb) ~DEL(B) ~PRINT~REV~PRINT"

stampa attesa:

listInput = "ADD(1) ~ ADD(a) ~ add(B) ~ ADD(B) ~ ADD ~ ADD(9) ~PRINT~SORT(a)~PRINT~DEL(bb) ~DEL(B) ~PRINT~REV~PRINT"

Comando Corrente	ADD(1)	ADD(a)	add(B)	ADD(B)	ADD	ADD(9)	PRINT	SORT(a)	PRINT	DEL(bb)	DEL(B)	PRINT	REV	PRINT
Elementi in Lista	1	1a	1a	1aB	1aB	1aB9	1aB9	1aB9	1aB9	1aB9	1a9	1a9	9a1	9a1

risultato:

```
Numero di comandi validi
13
Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Non Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Non Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido
1aB9

Controllo formattazione
Comando Non Valido

Controllo formattazione
Comando Valido
1aB9

Controllo formattazione
Comando Non Valido

Controllo formattazione
Comando Valido

Elemento trovato ed eliminato

Controllo formattazione
Comando Valido
1a9
```

```
Controllo formattazione
Comando Valido

Lista invertita correttamente

Controllo formattazione
Comando Valido
9a1

Programma terminato
Program exited with code: 0
```

test3:

stringa :listinput: .string "ADD(b) ~ ADD(3) ~ ADD(A) ~ ADD(@) ~ PRINT ~ SORT ~ PRINT"

stampa attesa:

inserimento di 4 nodi con caretteri diversi

Print dopo le add b3A@

print dopo sort : @3bA

risultato:

```
Numero di comandi validi
6
Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido
b3A@

Controllo formattazione
Comando Valido
```

```
Controllo formattazione
Comando Valido
@3bA

Programma terminato
Program exited with code: 0
```

test4:

stringa :listinput: .string "ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(9) ~ PRINT ~ DEL(a) ~ PRINT ~ REV ~ PRINT"

stampa attesa:

inserimento di 4 nodi : a a B 9

Print1 dopo le add: aaB9

DEL(a)

Print2: B9

REV

Print3 dopo REV: 9B

risultato:

```
Numero di comandi validi
8
Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido

Controllo formattazione
Comando Valido
aaB9

Controllo formattazione
Comando Valido

Elemento trovato ed eliminato
```

```
Controllo formattazione
Comando Valido
B9

Controllo formattazione
Comando Valido

Lista invertita correttamente

Controllo formattazione
Comando Valido
9B

Programma terminato
Program exited with code: 0
```

test5:

stringa: listinput: .string "ADD(x) ~ DEL(x) ~ ADD(y) ~ PRI ~ ADD(z) ~ ADD(1) ~ ADD(2) ~ ADD(3) ~ ADD(4) ~ ADD(5) ~ ADD(6) ~ ADD(7) ~ ADD(8) ~ ADD(9) ~ ADD(a) ~ ADD(b) ~ ADD(c) ~ ADD(d) ~ ADD(e) ~ ADD(f) ~ ADD(g) ~ ADD(h) ~ ADD(i) ~ ADD(j) ~ ADD(k) ~ ADD(l) ~ ADD(m) ~ ADD(n) ~ ADD(o) ~ ADD(p) ~ ADD(q)"

stampa attesa:

inserimento di 31 nodi : porta a errore

e comando non valido PRI che porta anche esso a errore

errore nell'esecuzione

risultato:

```
Numero di comandi maggiore o uguale a 30

Programma terminato
Program exited with code: 0
```